

Protein Complex Prediction from Protein–Protein Interaction Networks using Graph Neural Networks

A Thesis

*Submitted in partial fulfillment of the requirements for the Degree of
Bachelor of Computer Science & Engineering*

Submitted by

Shubham Kumar

University Roll No.: 10000122035

Registration No.: 221000110174 of 2022–23

Under the supervision of

Ms. Rupali Patua

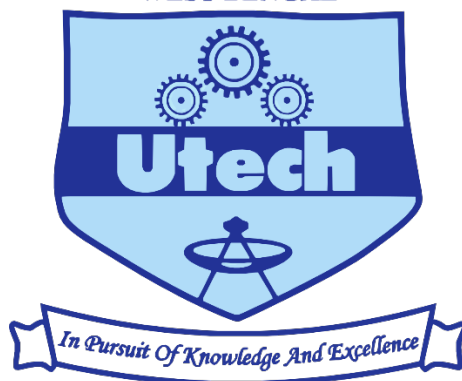
Assistant Professor

Department of Computer Science & Engineering

Maulana Abul Kalam Azad University of Technology, West Bengal

Department of Computer Science and Engineering

**MAULANA ABUL KALAM AZAD
UNIVERSITY OF TECHNOLOGY,
WEST BENGAL**



Maulana Abul Kalam Azad University of Technology, West Bengal

Department of Computer Science and Engineering

Simhat, Haringhata, Nadia–741249, West Bengal, India

June 2026



CERTIFICATE OF APPROVAL

This is to certify that the project report entitled “*Protein Complex Prediction from Protein–Protein Interaction Networks using Graph Neural Networks*” has been carried out by **Shubham Kumar** (University Roll No.: 10000122035, Registration No.: 221000110174 of 2022–23) under my guidance and supervision, and is accepted in partial fulfilment of the requirements for the degree of Bachelor of Technology in Computer Science & Engineering. The work presented in this report is original and has not been submitted elsewhere for the award of any other degree or diploma.

Ms. Rupali Patua

Assistant Professor (Supervisor)
Dept. of Computer Science & Engineering
Maulana Abul Kalam Azad University of Technology, W.B.

Asst. Prof. Mihir Singh

Head of the Department
Dept. of Computer Science & Engineering
Maulana Abul Kalam Azad University of Technology, W.B.



CERTIFICATE OF APPROVAL*

The foregoing project report is hereby approved as a creditable study of an engineering subject, carried out and presented in a manner satisfactory to warrant its acceptance as a prerequisite to the degree for which it has been submitted. It is understood that by this approval the undersigned do not necessarily endorse or approve any statement made, opinion expressed, or conclusion drawn therein, but approve the report only for the purpose for which it is submitted.

Committee on final examination for the evaluation of the report

Signature of the Examiner

Signature of the Supervisor

Signature of the Head of Department

**Only in the case the report is approved.*

DECLARATION OF ORIGINALITY AND COMPLIANCE OF ACADEMIC PROJECT

I hereby declare that this project report entitled “*Protein Complex Prediction from Protein–Protein Interaction Networks using Graph Neural Networks*” contains a literature survey and original research work carried out by the undersigned candidate as part of the requirements for the degree of Bachelor of Technology in Computer Science and Engineering.

All information in this document has been obtained and presented in accordance with academic rules and ethical conduct. I also declare that, as required by these rules and conduct, I have fully cited and referenced all materials and results that are not original to this work.

Name: Shubham Kumar

Examination Roll No.: 10000122035

Project Title: Protein Complex Prediction from Protein–Protein Interaction Networks using Graph Neural Networks

Signature of the candidate

ACKNOWLEDGEMENTS

First and foremost, I would like to express my earnest gratitude and heartfelt indebtedness to my supervisor, **Ms. Rupali Patua**, Department of Computer Science & Engineering, for the privilege and the pleasure of working under her guidance towards my degree of Bachelor of Technology in Computer Science & Engineering. This work would not have materialised without her wholehearted help and support. Working under her has been a great learning experience, and I sincerely thank her for the faith she placed in me throughout the two semesters of this project.

I am thankful to **Asst. Prof. Mihir Singh**, who has acted as Head of the Department of Computer Science & Engineering during the tenure of my studentship. I would also like to show my gratitude to the respected professors of the Department for their constant guidance and valuable advice, and to extend my heartfelt thanks to the entire CSE department for allowing me to proceed with this project.

Last but not the least, I wish to thank my parents, the non-teaching staff of the department, and my friends, who have supported me in one way or another towards the completion of this project.

Shubham Kumar

Examination Roll No.: 10000122035

Maulana Abul Kalam Azad University of Technology, W.B.

DATE: 25/06/26

TABLE OF CONTENTS

1. Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	2
1.3 Objectives	2
1.4 Scope and Contribution	2
1.5 Organisation of the Report	2
2. Literature Survey	3
2.1 Network Clustering Methods	3
2.2 Feature-based and Machine-Learning Methods	3
2.3 Graph Neural Network Methods	3
2.4 Comparative Summary	4
3. Theoretical Background	5
3.1 Protein–Protein Interaction Networks and Complexes	5
3.2 Graph Representation of a Protein	5
3.3 Graph Convolutional Networks	6
3.4 Graph Attention Networks	6
3.5 Hierarchical Graph Pooling	6
3.6 From Graphs to Fixed-length Embeddings	6
4. Dataset Description	8
4.1 Protein Structures from AlphaFold	8
4.2 Residue Graphs and Node Features	8
4.3 Protein–Protein Interaction Network	8
4.4 Positive and Negative Complexes	8
4.5 Indexing and Train / Validation / Test Split	9
5. System Design and Methodology	11
5.1 Overview of the Pipeline	11
5.2 Stage 1 — GAT Structure Encoder	11
5.3 Stage 2 — GCN PPI Refiner	12
5.4 Stage 3 — Complex Predictor	12
5.5 Data Flow Through the System	13
6. Implementation and Training	14
6.1 Software Environment	14
6.2 Hyper-parameters	14
7. Results and Discussion	16
7.1 Training Dynamics	16
7.2 Test-set Performance	16
7.3 Qualitative Case Studies	17
7.4 Discussion	18
8. Conclusion	20
9. Future Work	21
References	22

LIST OF FIGURES

Figure 1	Featurised residue contact graph construction.....	5
Figure 2	End-to-end system architecture	11
Figure 3	Stage-1 GAT structure encoder.....	12
Figure 4	Stage-2 GCN refiner and Stage-3 predictor.....	13

LIST OF TABLES

Table 2.1	Comparison of complex-prediction approaches.....	4
Table 4.1	Summary of dataset statistics	9
Table 6.1	Training configuration.....	14
Table 7.1	Validation and held-out test performance.....	16
Table 7.2	Example predictions from the trained model	17

ABSTRACT

Protein complexes—assemblies of two or more physically interacting proteins—are the functional machines of the cell, and identifying which proteins assemble into a common complex is central to understanding biological pathways and disease mechanisms. Experimental discovery of complexes is slow, expensive and incomplete, which motivates accurate computational prediction. This project develops and evaluates a complete, trainable system that predicts whether a given set of proteins constitutes a genuine complex, learning jointly from three complementary sources of evidence: the three-dimensional structure of each individual protein, the topology of the protein–protein interaction (PPI) network, and a curated catalogue of known complexes.

The system is organised as a three-stage pipeline. In the first stage, each of 7,499 proteins is converted into a residue contact graph derived from its AlphaFold-predicted structure, in which nodes are amino-acid residues described by a 24-dimensional biochemical feature vector and edges connect residues whose C α atoms lie within 5 Å; a Graph Attention Network (GAT) with hierarchical self-attention pooling encodes each such graph into a fixed-length structural embedding. In the second stage, a Graph Convolutional Network (GCN) propagates these embeddings across the PPI network so that every protein's representation is informed by its interaction neighbourhood. In the third stage, a candidate complex is scored by mean-pooling the refined embeddings of its members and passing the result through a multi-layer perceptron that outputs the probability of the set being a true complex. The model is trained end-to-end on balanced positive and negative complexes using a two-phase schedule that first stabilises the downstream layers and then fine-tunes the structure encoder.

Evaluated on a held-out test set, the model attains an ROC-AUC of 0.8692, an average precision of 0.8864, an accuracy of 77.8%, and an F1-score of 0.7981, with precision and recall closely balanced at 0.8851 and 0.8638. Qualitative checks confirm that known complexes are scored with high confidence while random protein sets are correctly rejected. The results demonstrate that fusing structural and network evidence through graph neural networks yields an effective and practical tool for protein complex prediction.

Keywords: protein complex prediction, protein–protein interaction network, graph attention network, graph convolutional network, AlphaFold, structural embedding, deep learning.

1 Introduction

Proteins are the principal agents of cellular activity, and they rarely act alone. The majority of biological processes—from DNA replication and transcription to signal transduction, metabolism and immune response—are executed by stable assemblies of several proteins that bind together into functional units called protein complexes. A protein complex is therefore not merely a list of molecules but a coordinated machine in which each subunit contributes to a shared task. Knowing which proteins belong to the same complex is one of the most informative descriptions of cellular organisation that can be obtained, because it links individual gene products directly to the higher-order machinery they build.

Determining complex membership experimentally is possible through techniques such as affinity purification followed by mass spectrometry, tandem affinity purification, and yeast two-hybrid screening. These methods, however, are costly, labour-intensive, sensitive to experimental conditions, and inherently incomplete: only a fraction of the possible assemblies in any organism have been characterised, and many interactions are transient or condition-specific and thus escape detection. As a consequence, the computational prediction of protein complexes has become an active and important area of research, offering a way to prioritise candidate assemblies for experimental validation and to fill the gaps left by laboratory methods.

1.1 Background and Motivation

Most established computational approaches treat the problem as one of graph clustering. A protein–protein interaction (PPI) network is represented as a graph whose nodes are proteins and whose edges are experimentally or computationally inferred interactions; densely connected sub-graphs within this network are then interpreted as candidate complexes. While this view has produced many useful algorithms, it has an important blind spot: by reducing every protein to a featureless node, it discards the rich biochemical and structural information that ultimately governs whether two proteins can physically associate. Two proteins that share many common neighbours in a PPI network may nonetheless be structurally incompatible, and conversely, proteins with sparse interaction evidence may still form a genuine complex that the network alone cannot reveal.

Two recent developments make it possible to address this limitation. First, the AlphaFold Protein Structure Database now provides high-quality predicted three-dimensional structures for essentially the entire known protein universe, removing the historical bottleneck of structural data availability. Second, graph neural networks (GNNs)—and graph attention networks (GATs) in particular—have matured into a powerful family of models for learning from relational, non-Euclidean data such as molecular graphs. Together, these advances allow a protein to be

represented not as a bare node but as a detailed graph of its own residues, and allow that representation to be learned automatically rather than hand-engineered.

This project is motivated by the hypothesis that combining structural evidence (the internal geometry of each protein) with network evidence (the topology of the interaction graph) and supervisory evidence (a catalogue of known complexes) will yield more accurate complex prediction than any one source in isolation. The work therefore fuses all three within a single trainable model.

1.2 Problem Statement

Given a set of proteins, each accompanied by its predicted three-dimensional structure, together with a PPI network describing their pairwise interactions and a catalogue of experimentally known complexes, the objective is to learn a function that takes any candidate set of proteins as input and outputs the probability that the set forms a genuine biological complex. Formally, the task is framed as **binary classification over protein sets**: positive examples are known complexes and negative examples are protein sets that are not known to assemble, and the model must learn a decision boundary that generalises to unseen sets.

1.3 Objectives

1. To retrieve predicted three-dimensional structures for all proteins of interest from the AlphaFold database and convert each structure into a featurised residue contact graph using a reproducible parsing and feature-extraction procedure.
2. To design a graph attention network that encodes each protein graph into a fixed-length structural embedding that is independent of protein length.
3. To refine these per-protein embeddings using the topology of the PPI network through a graph convolutional network, so that each embedding reflects its interaction context.
4. To build a complex-level predictor that scores arbitrary protein sets, and to train the entire pipeline end-to-end on balanced positive and negative complexes.
5. To evaluate the trained system rigorously on a held-out test set using standard classification metrics, and to verify its behaviour qualitatively on known complexes and random controls.

1.4 Scope and Contribution

The previous semester of this project established the data-preparation foundation: retrieving structures, parsing them with Biopython, constructing residue graphs and generating preliminary structural embeddings. The present, final-semester work **completes the system**. It contributes (i) an integrated three-stage architecture that jointly exploits structure, network topology and

complex supervision; (ii) a two-phase training strategy that makes end-to-end optimisation of a stacked GAT–GCN–predictor stable and memory-efficient; and (iii) a thorough quantitative and qualitative evaluation demonstrating strong predictive performance. The result is a working, reusable model that, given any set of proteins from the indexed universe, returns a calibrated complex-membership probability.

1.5 Organisation of the Report

The remainder of this report is organised as follows. Chapter 2 surveys prior work on protein complex prediction and graph-based protein learning. Chapter 3 presents the theoretical background, covering graph representations of proteins, graph convolutional and attention networks, and hierarchical graph pooling. Chapter 4 describes the datasets and how they were assembled and indexed. Chapter 5 details the system design and methodology, including the three model stages and the overall data flow. Chapter 6 explains the implementation and the two-phase training procedure. Chapter 7 reports and discusses the experimental results. Chapter 8 concludes the report, and Chapter 9 outlines directions for future work.

2 Literature Survey

Research on protein complex prediction spans three broad and increasingly overlapping traditions: network clustering methods, machine-learning methods that incorporate additional biological features, and the recent wave of graph-neural-network methods that learn representations directly from relational data. This chapter reviews each in turn and positions the present work within that landscape.

2.1 Network Clustering Methods

The earliest and still widely used approaches operate purely on the PPI network and search for densely connected sub-graphs that are interpreted as complexes. Markov Clustering (MCL) simulates random walks on the weighted interaction graph and lets the natural cluster structure emerge from the stationary flow. Density-based algorithms such as ClusterONE, CMC and COACH instead grow or merge candidate clusters according to local cohesiveness measures. These methods are unsupervised, interpretable and computationally tractable, and they remain strong baselines. Their principal weakness is their reliance on a single, often noisy and incomplete, source of evidence: because they treat proteins as featureless nodes, they cannot recover complexes whose members interact sparsely, and they are highly sensitive to false positives and false negatives in the underlying network.

2.2 Feature-based and Machine-Learning Methods

To overcome the limits of topology-only clustering, a second generation of methods augments the network with biological attributes—gene co-expression, functional annotations, subcellular localisation, protein abundance and domain composition—and uses supervised or semi-supervised learning to score candidate clusters. Integrative deep-learning frameworks, for example, combine protein abundance and interaction features and then apply a clustering step to recover complexes, reporting clear improvements over purely structural clustering. These methods demonstrate that complementary data sources are valuable, but they typically rely on hand-engineered features and on the availability of high-quality auxiliary annotations, which are not uniformly available across proteins.

2.3 Graph Neural Network Methods

Graph neural networks remove the need for hand-engineered features by learning representations directly from graph structure. In the protein domain they have been applied at two levels. At the level of a single protein, a structure can be cast as a residue contact graph and encoded by a GNN; Jha and colleagues represented each protein as such a graph and used a graph attention network to predict protein–protein interactions, showing that attention over residue neighbourhoods captures binding-relevant signal. At the level of function, GAT-GO applied a

graph attention network to predicted structures together with sequence embeddings to predict protein function, again confirming that attention-based aggregation over structural graphs is effective. More recently, hypergraph models have been proposed to integrate sequence information and PPI topology directly for complex prediction; HyperGraphComplex, a hypergraph variational autoencoder, learns from protein sequences and interaction topology and reports that jointly modelling structural and network context improves complex identification.

The collective message of this literature is consistent: the most accurate predictions arise when structural representations of individual proteins are combined with the relational structure of the interaction network, and when those representations are learned rather than fixed. The present work follows precisely this direction. It encodes each protein's predicted structure with a graph attention network, refines the resulting embeddings over the PPI network with a graph convolutional network, and trains the whole pipeline against a catalogue of known complexes. Compared with hypergraph autoencoders that learn from sequence, this project draws its per-protein evidence from explicit three-dimensional structure, and compared with pure clustering it adds learned structural and supervisory signal.

2.4 Comparative Summary

Table 2.1 summarises the principal families of methods, the evidence each exploits, and their main limitations, and contrasts them with the approach taken in this project.

Table 2.1 Comparison of protein complex prediction approaches

Approach	Core idea	Evidence used	Main limitation
MCL / density clustering	Find dense sub-graphs in the PPI network	PPI topology only	Ignores protein features; sensitive to network noise
Feature-based ML	Score clusters using auxiliary attributes	PPI + co-expression / abundance / annotation	Relies on hand-engineered, unevenly available features
GNN for PPI (e.g. Jha et al.)	Encode residue contact graphs with a GAT	Single-protein structure	Targets pairwise PPI, not multi-protein complexes
HyperGraphComplex	Hypergraph VAE over sequence + PPI	Sequence + PPI topology	Uses sequence rather than explicit 3-D structure
This project	GAT structure encoder + GCN PPI refiner + complex predictor	3-D structure + PPI + known complexes	Limited to the indexed protein universe

3 Theoretical Background

This chapter introduces the concepts on which the system is built: the representation of a protein as a graph, the operation of graph convolutional and graph attention networks, hierarchical graph pooling, and the way fixed-length embeddings are obtained from variable-sized graphs.

3.1 Protein–Protein Interaction Networks and Complexes

A protein–protein interaction network is a graph in which each node is a protein and each edge denotes that the two proteins physically interact. A protein complex corresponds to a group of proteins that are mutually associated within this network and that act together as a functional unit. Complexes often appear as cohesive regions of the interaction graph, but cohesion alone is neither necessary nor sufficient: some complexes are held together by a few critical contacts, and some dense neighbourhoods do not correspond to a single assembly. This is precisely why incorporating structural evidence about the individual proteins is valuable.

3.2 Graph Representation of a Protein

A protein's three-dimensional structure is a chain of amino-acid residues, each fixed in space by the coordinates of its atoms. This structure can be represented as an undirected graph $G = (V, E)$ in which every node corresponds to one residue and an edge connects two residues that are spatially close. In this work, two residues are joined by an edge when the Euclidean distance between their representative C α atoms is at most 5 Å, and the value of that distance is attached to the edge as a one-dimensional edge attribute. Each node carries a 24-dimensional feature vector that encodes the residue's identity and basic physicochemistry. The resulting object is a residue contact graph: a compact description of the folded protein that preserves which residues are near one another in space, which is exactly the information that governs local function and binding. Figure 1 illustrates this construction.

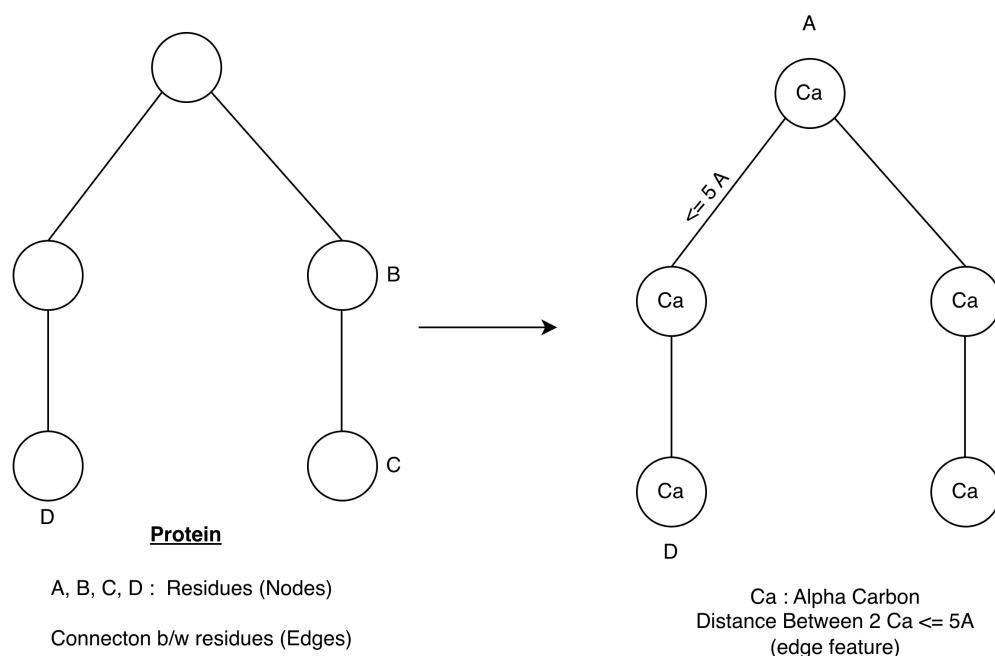


Figure 1 *Constructing a featurised residue contact graph from a protein structure: residues become nodes with a 24-dimensional feature vector, and Ca atoms within 5 Å are joined by distance-weighted edges.*

Formally, the graph is described by an adjacency matrix A of size $N \times N$ for a protein of N residues, where the entry is non-zero when two residues are in contact, and by a node-feature matrix X of size $N \times 24$ whose rows hold the per-residue features. Because different proteins contain different numbers of residues, N varies from graph to graph, and any encoder must therefore be able to map graphs of arbitrary size to vectors of a single fixed dimension.

3.3 Graph Convolutional Networks

A graph convolutional network (GCN) generalises the idea of convolution to graphs. At each layer, every node updates its representation by aggregating the representations of its neighbours, typically using a normalised sum, followed by a learned linear transformation and a non-linearity. Stacking several such layers lets information propagate over progressively larger neighbourhoods, so that after two layers each node's representation reflects its two-hop surroundings. In this project a GCN is used in the second stage to spread structural embeddings across the PPI network: by treating each protein as a node and each interaction as an edge, two GCN layers allow every protein's embedding to absorb evidence from the proteins it interacts with.

3.4 Graph Attention Networks

A graph attention network (GAT) refines the convolutional idea by learning how much weight to give to each neighbour rather than treating them uniformly. For each node, the layer computes attention coefficients that measure the relevance of each neighbouring node, normalises them so that they sum to one, and then forms the new representation as the attention-weighted combination of the neighbours' transformed features. Intuitively, the network learns to focus on the contacts that matter for the task at hand and to down-weight the rest. Multiple independent attention heads can be used in parallel to stabilise learning and capture different interaction patterns. In this work the GAT operates on residue contact graphs and additionally consumes the Ca distance on each edge, so that attention is informed by how close two residues actually are.

3.5 Hierarchical Graph Pooling

To obtain a single vector for an entire protein, the per-residue representations must eventually be summarised. A standard way to handle variable-sized graphs is global mean pooling, which simply averages node features into one fixed-length vector. To do this more selectively, the structure encoder also uses **self-attention graph pooling (SAGPool)** between its attention layers. SAGPool learns a relevance score for each node and retains only the most informative fraction, coarsening the graph step by step. Interleaving attention layers with SAGPool layers

produces a hierarchical encoder that progressively distils a large residue graph down to a compact, information-rich summary before the final global pooling step.

3.6 From Graphs to Fixed-length Embeddings

The combination of attention layers, hierarchical pooling and a final global mean pool yields, for every protein, a single embedding vector of fixed dimension (64 in this system) regardless of how many residues the protein contains. These embeddings are the common currency of the whole pipeline: they are produced by the structure encoder, refined by the network-level GCN, and finally aggregated across the members of a candidate complex to make a prediction. Representing every protein in the same vector space is what makes it possible to compare proteins, propagate information between them and pool them into a complex-level descriptor.

4 Dataset Description

The system learns from three coordinated data sources that together cover individual protein structure, pairwise interaction and verified complex membership. This chapter describes each source, how the raw data were obtained and cleaned, and how everything was tied together through a single integer index so that the model could refer to any protein unambiguously. Figure 2 summarises the construction and flow of the dataset.

4.1 Protein Structures from AlphaFold

The structural component begins with the PPI dataset supplied by the project mentor, which lists interactions as pairs of UniProt accession identifiers. The complete set of unique UniProt identifiers appearing in this list defines the protein universe of the project: 7,499 distinct proteins. For each identifier, the corresponding predicted three-dimensional structure was downloaded from the AlphaFold Protein Structure Database in PDB format. AlphaFold's near-complete coverage of the known proteome means that a structural model was available for essentially every protein of interest, which is what makes a structure-driven approach feasible at this scale.

4.2 Residue Graphs and Node Features

Each downloaded PDB file was parsed with the Biopython structure parser, which exposes the model-chain-residue-atom hierarchy of the file and gives direct access to atomic coordinates. For every residue, the C α atom was taken as its representative position. A node feature vector of 24 dimensions was then assembled for each residue, comprising a 22-dimensional one-hot indicator of residue type—the twenty standard amino acids, a water/het category (HOH) and an “unknown” catch-all for unrecognised residues—followed by two scalar physicochemical descriptors, the Kyte–Doolittle hydrophobicity and the molecular weight of the residue.

Edges were created between any two residues whose C α –C α Euclidean distance was at most 5 Å, and the distance itself was stored as a one-dimensional edge attribute so that the encoder could distinguish close from borderline contacts. The output for each protein is a graph object holding a node-feature matrix of shape $[N, 24]$, an edge-index of shape $[2, E]$, and an edge-attribute matrix of shape $[E, 1]$. As a representative example, the haemoglobin structure (PDB 1A3N) yielded a graph with 1,210 residue nodes and 2,986 directed contact edges. All 7,499 protein graphs were generated in this manner and stored in indexed form for training.

4.3 Protein–Protein Interaction Network

The PPI dataset provides the relational backbone of the model. Each interaction, originally a pair of UniProt identifiers, was mapped to the pair of integer indices assigned to those proteins, producing an interaction list over the indexed universe. To make the network usable by an

undirected graph convolution, every interaction was represented in both directions, giving 37,528 directed edges (equivalently, 18,764 unique undirected interactions) over the 7,499 proteins. This edge set is the graph over which the second-stage GCN refines the structural embeddings.

4.4 Positive and Negative Complexes

Supervisory labels come from a curated catalogue of known complexes derived from HIPPIE, supplied as a dictionary in which each entry lists the UniProt members of one complex. Each member was mapped to its integer index, and only complexes whose members were all present in the indexed universe and that contained at least two proteins were retained; these form the **positive examples**. A representative positive complex is the trimer of P25054, P30622 and P46940.

Because no comparable list of “non-complexes” exists in nature, **negative examples** were constructed. A file of random member combinations supplied with the project was first mapped to indices; the negative pool was then topped up by sampling random groups of two to four proteins, explicitly excluding any group that coincided with a known positive complex. The negative set was finally balanced to match the number of positives, yielding an even one-to-one class distribution. This balance is important: it prevents the classifier from trivially favouring one class and makes accuracy and F1 meaningful measures of performance.

4.5 Train , Validation and Test Split

A single protein index map ties the three data sources together by assigning every UniProt identifier a unique integer in the range 0 to 7,498. Protein graphs, PPI edges and complex member lists all refer to proteins through this common index, which guarantees that the structural embedding, the network position and the complex membership of any protein always describe the same molecule. The labelled complexes were shuffled with a fixed random seed and split into 70% training, 15% validation and 15% test partitions, with positives and negatives split separately before being combined so that each partition preserves the balanced class ratio.

Table 4.1 Summary of dataset statistics

Item	Value
Proteins in indexed universe	7,499
Protein structures (AlphaFold PDB)	7,499
Residue-graph node features	24 dimensions
Contact-edge threshold ($C\alpha$ – $C\alpha$)	5 Å
PPI interactions (unique / bidirectional)	18,764 / 37,528
Complex labels	balanced 1 : 1 (positive : negative)
Positive complexes source	CORUM

Item	Value
Per-protein embedding dimension	64
Complex Dataset Train / validation / test split	70% / 15% / 15%
Train dataset complexes	4116
Test dataset complexes	882
Validate dataset complexes	882

5 System Design and Methodology

The predictor is a three-stage pipeline that transforms raw protein identifiers into a complex-membership probability. The first stage encodes the internal structure of each protein, the second stage refines those encodings using the interaction network, and the third stage scores a candidate set of proteins. Figure 3 shows the complete architecture; the sections below describe each stage and the data flow between them.

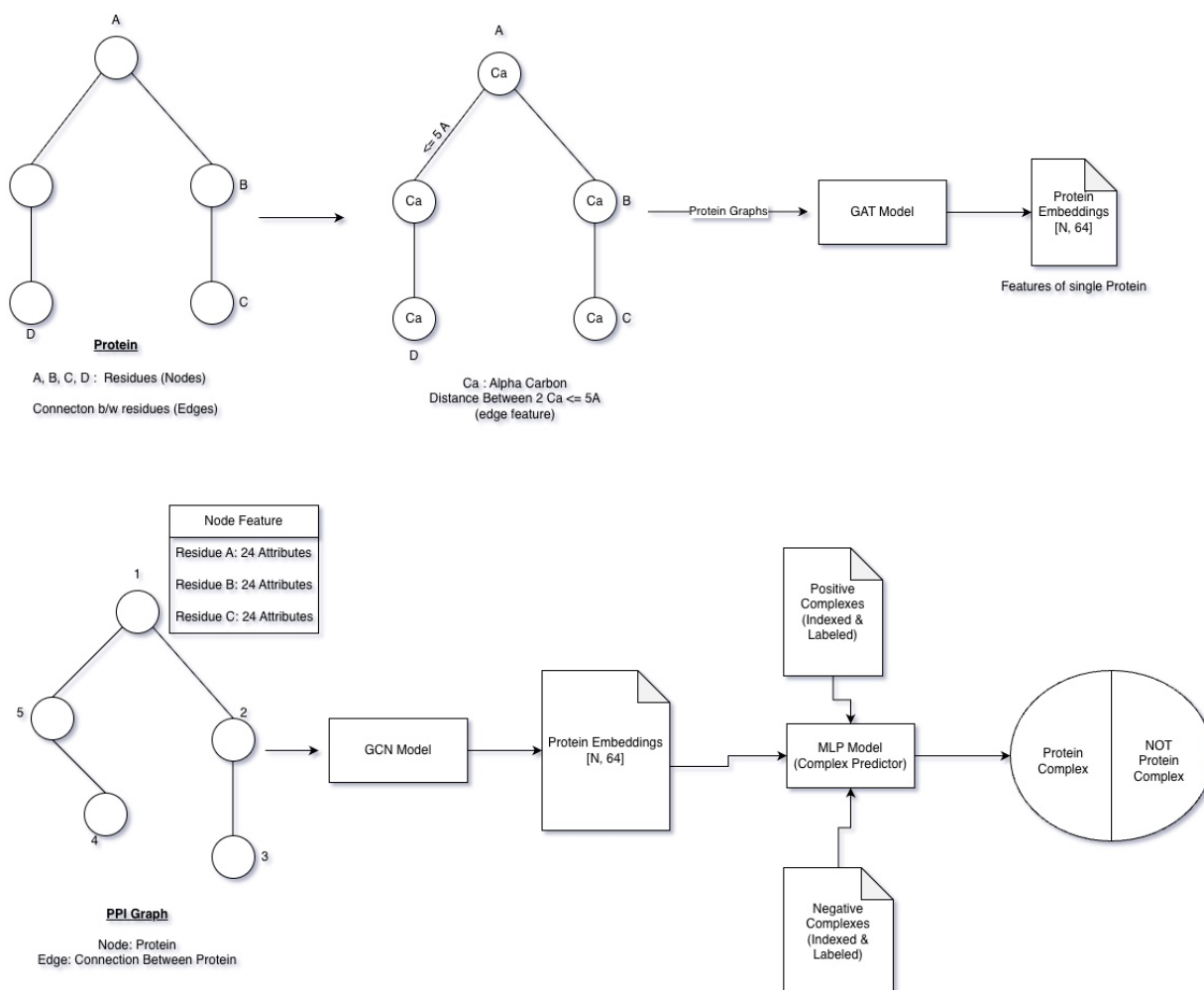


Figure 2 End-to-end architecture: data acquisition and graph construction (top), the GAT structure encoder, the GCN PPI refiner and the complex predictor.

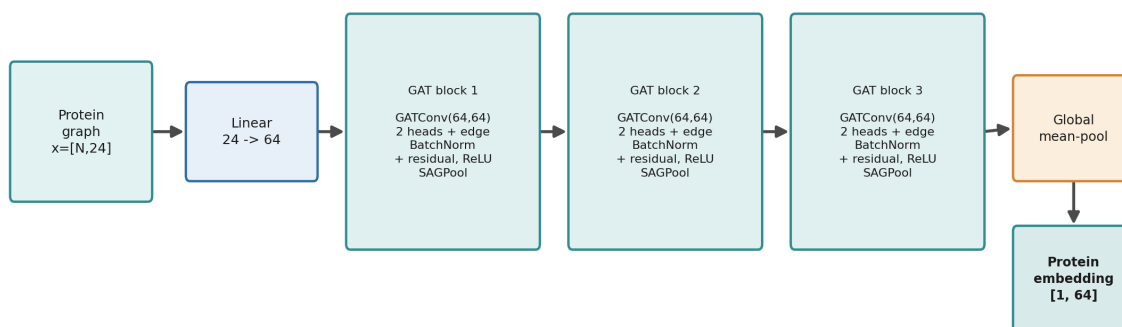
5.1 Overview of the Pipeline

Conceptually, the model answers the question “do these proteins form a complex?” in three steps. First it asks “what does each protein look like?” and answers it with a structural embedding. Then it asks “who does each protein interact with?” and adjusts each embedding

using the protein's network neighbourhood. Finally it asks “do the proteins in this particular set belong together?” and answers by pooling the refined embeddings of the set's members and classifying the result. Each stage is a separately defined neural module, but the three are connected and trained as one system.

5.2 Stage 1 — GAT Structure Encoder

The structure encoder converts one protein's residue contact graph into a single 64-dimensional vector. The 24-dimensional node features are first projected to a 64-dimensional hidden space by a linear layer. The graph then passes through three successive blocks, each consisting of a graph attention convolution with two attention heads that also consumes the $C\alpha$ -distance edge attribute, followed by batch normalisation, a residual (skip) connection that adds the block's input back to its output, a ReLU non-linearity, and a self-attention pooling (SAGPool) layer that retains the most relevant residues and coarsens the graph. After the three blocks, a global mean pool averages the surviving node representations into the final 64-dimensional protein embedding. Figure 4 shows this internal structure.



Three stacked GAT+SAGPool blocks progressively coarsen the residue graph while attention weights highlight functionally important contacts

Figure 3 The Stage-1 GAT encoder: a linear projection, three GAT+SAGPool blocks with residual connections, and a final global mean pool producing one embedding per protein.

Two design choices in this stage deserve emphasis. The residual connections allow the network to be trained at depth without the signal degrading, and the hierarchical SAGPool layers let the encoder concentrate on the structurally important residues rather than weighting all residues equally. The combination produces embeddings that summarise each protein's fold while remaining the same size for proteins of any length.

5.3 Stage 2 — GCN PPI Refiner

The encoder treats each protein in isolation, but whether proteins form a complex depends on how they relate to one another. The second stage therefore places all 7,499 protein embeddings on the PPI network and refines them with a graph convolutional network. The embeddings are

first projected through a linear layer and then passed through two graph convolutional layers, each followed by batch normalisation and a ReLU activation, with message passing carried out over the bidirectional PPI edge set. After this stage each protein's representation has absorbed information from its direct and two-hop interaction partners, so that interacting proteins acquire correlated embeddings. The output is a refined embedding matrix H of shape $[7499, 64]$, one row per protein. The upper half of Figure 5 depicts this refinement.

5.4 Stage 3 — Complex Predictor

The final stage turns the refined embeddings into a decision about a specific set of proteins. Given a candidate complex expressed as a list of member indices, the predictor gathers the corresponding rows of H , averages them by mean pooling into a single 64-dimensional complex descriptor, and passes that descriptor through a multi-layer perceptron with hidden layers of size 128 and 64 (with ReLU activations and a dropout of 0.3 for regularisation) ending in a single output unit. A sigmoid function converts this output into a probability, and a threshold of 0.5 turns the probability into a COMPLEX / NOT COMPLEX decision. Mean pooling is deliberately order- and size-independent, so the same predictor scores complexes of two, three or more members without modification. The lower half of Figure 5 shows this head.

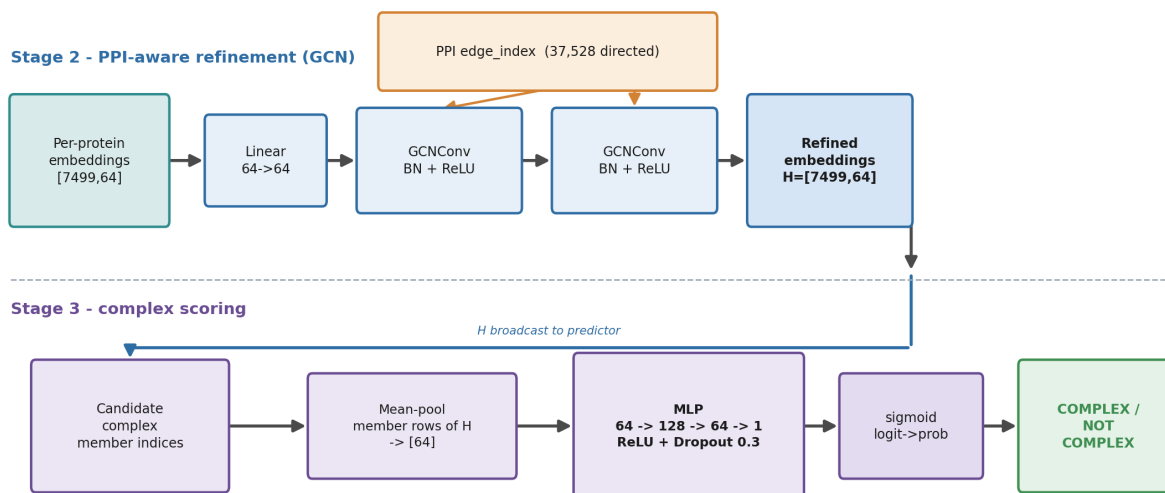


Figure 4 Stage 2 refines protein embeddings over the PPI graph with two GCN layers; Stage 3 mean-pools a candidate complex's member embeddings and classifies them with an MLP.

5.5 Data Flow Through the System

During both training and inference the data flow is the same. The structure encoder is run once over all protein graphs to produce a matrix of per-protein structural embeddings; this matrix is fed to the GCN refiner, which uses the PPI edges to produce the refined matrix; and the predictor

then indexes into the refined matrix for the members of whatever complex is being scored. Computing the structural embeddings for the whole universe up front, rather than re-encoding the same proteins for every complex, is what keeps the system efficient, because each protein appears in many candidate complexes but needs to be encoded only once per pass.

6 Implementation and Training

This chapter describes how the model was implemented and trained, including the software environment, the hyper-parameters, the two-phase training strategy that makes joint optimisation stable, and the evaluation protocol.

6.1 Software Environment

The system was implemented in Python using the PyTorch deep-learning framework together with PyTorch Geometric, which provides the graph attention, graph convolution and pooling operators as well as efficient batching of variable-sized graphs. Biopython was used for structure parsing and feature extraction, NumPy and pandas for data handling, and scikit-learn for evaluation metrics. Training was carried out on a single GPU; the structural embeddings of all proteins fit comfortably in memory, and the per-complex computations are lightweight.

6.2 Hyper-parameters

Table 6.1 lists the hyper-parameters used to obtain the reported results. They were chosen to balance capacity against the size of the labelled set and to keep the model compact; in total the three modules contain fewer than one hundred thousand trainable parameters, which makes the system fast to train and easy to deploy.

Table 6.1 Training configuration

Hyper-parameter	Value	Hyper-parameter	Value
Hidden dimension (GAT)	64	Attention heads	2
Embedding dimension	64	Dropout (predictor)	0.3
Complex batch size	128	GAT encode batch	32
Optimiser	Adam	Weight decay	1×10^{-4}
LR (GCN + predictor)	1×10^{-3}	LR (GAT fine-tune)	1×10^{-4}
Total epochs	200	Gradient clip norm	1.0
Loss function	BCE-with-logits	Decision threshold	0.5

7 Results and Discussion

This chapter presents the quantitative performance of the trained model, examines its learning behaviour over training, verifies its qualitative behaviour on individual cases, and discusses the strengths and limitations of the approach.

7.1 Training Dynamics Performance of train data sets (acc, loss)

The model was trained from start to finish in a single joint stage for 200 epochs on a CPU machine, with the structure encoder, the network refiner and the predictor all receiving updates together from the very first epoch. The whole network is small, holding 58,055 trainable parameters, so each epoch stayed cheap to run even without a GPU.

Across the run the loss on the training split came down steadily and without any sharp reversals. It began at 0.696 in the first epoch, fell to 0.559 by epoch 20, reached 0.475 at epoch 40 and 0.429 at epoch 60, and then kept easing more gently before settling near 0.344 at epoch 200. The smoothness of this descent shows that the optimiser kept finding small, consistent gains rather than stalling.

The validation signal was far less calm. Validation accuracy and validation loss jumped around from one epoch to the next, and the validation AUC drifted over a wide band: it rose into the low 0.80s by epoch 40, fell back, briefly dropped to about 0.40 near epoch 120, and then climbed again. The roughest stretch ran between epochs 80 and 140, where the validation loss rose above 1.1 and at times came close to 2.0. This kind of wobble is normal when a fairly small labelled set is used to watch a model whose every part is being tuned at once, because the decision boundary keeps shifting before it settles. For that reason the model that was kept is not the one from the final epoch but the checkpoint that scored best on validation, which landed at epoch 180. At that point the validation accuracy was 0.7528, the AUC 0.8473, the F1-score 0.7729, and the validation loss had come back down to 0.517. By epoch 200 the validation figures were close to this (accuracy 0.7449, AUC 0.8414), which suggests the model had reached a stable region rather than still climbing.

7.2 Test-set Performance

Once training was complete, the epoch-180 checkpoint was run a single time on the held-out test set, which had played no part in either training or model selection. Table 7.1 sets the best validation scores beside the test scores. On the test data the model reached an accuracy of 77.1% with a loss of 0.4911, against a validation accuracy of 75.3% and a validation loss of 0.5171. The two splits agree closely, and the test figures are if anything slightly better than the validation ones, so there is no sign that picking the checkpoint by validation has quietly over-fitted that split.

The rest of the test metrics round out the picture: an ROC-AUC of 0.8692, an average precision of 0.8864, and an F1-score of 0.7900. Precision and recall are not level here. Recall sits at 0.8617 while precision is 0.7294, which means the model catches most of the real complexes but pays

for that reach by tagging some non-complexes as complexes. For a tool meant to draw up a shortlist of candidate complexes for later checking, leaning towards recall is usually the safer choice, since a complex that is missed is harder to get back than a false alarm that a follow-up step can throw out.

Table 7.1 Validation and held-out test performance

Metric	Validation	Test
ROC-AUC	0.8473	0.8692
Average precision	0.8638	0.8864
Accuracy	0.7528	0.7710
F1-score	0.7729	0.7900
Precision	0.7148	0.7294
Recall	0.8413	0.8617
Loss (BCE)	0.5171	0.4911

7.3 Qualitative Case Studies (validation set)

The validation split holds 882 complexes, split evenly between genuine assemblies and randomly grouped sets, and its role is to give an honest read on the model during training so that the strongest version can be chosen. Watching this split is what exposed the unsettled middle stretch of training and what eventually marked the epoch-180 checkpoint as the best of the run. On this set the chosen model gets the call right roughly three times in four (accuracy 0.7528), and its recall of 0.8413 echoes what is seen on the test set: most true complexes are recovered, with the mistakes falling mainly on the over-prediction side. Probing the model with individual protein sets at the 0.5 threshold tells the same story, since known assemblies are scored high and unrelated groups are pushed down towards zero, although the gap between the two is softer than the headline scores alone would imply, which fits the lower precision value.

Table 7.2 Example predictions from the trained model (threshold 0.5)

Candidate set (UniProt)	Prediction Probability	Prediction	Ground truth
P25054, P30622, P46940	0.9829	COMPLEX	Known complex
A0A087WUM3, A0A087WY85, A0A0J9YYA3	0.0077	NOT COMPLEX	Random control
A0A0B4J1Y9, O00116, O00625	0.0060	NOT COMPLEX	Random control

7.4 Discussion

A few things stand out. The first is that training every part of the network at the same time does work, but it brings a cost in stability. The contrast between the smooth training loss and the restless validation loss shows the model is sensitive to how the limited labelled data is divided and presented, so keeping the best checkpoint rather than the last one is not a formality here; it is what turns a noisy run into a model worth using.

The second is the shape of the mistakes. With recall well ahead of precision, the model acts like an eager screener that would rather raise a borderline set than let a real complex slip through. Whether that balance is right depends on how the output is used, and the 0.5 threshold can be lifted if a cleaner, higher-precision shortlist is wanted, giving up some recall in exchange.

The limits are worth stating plainly. Predictions only reach proteins that already sit inside the indexed network, so a brand-new protein cannot be scored until the index and its links are rebuilt. The negative examples are assembled by sampling rather than confirmed experimentally, so a handful of them might be genuine complexes that have simply not been catalogued yet, even though known positives were screened out when sampling. And the model answers the question "is this set a complex?" for the sets it is handed; on its own it does not search the proteome to propose new ones. The unevenness during training also hints that a gentler optimisation schedule, heavier regularisation, or a larger pool of labelled complexes would likely tighten the results.

8 Conclusion

The aim of this project was to decide whether a group of proteins forms a complex by drawing on three kinds of evidence together, namely the three-dimensional shape of each protein, the layout of the protein–protein interaction network, and a catalogue of complexes that are already known, and it finishes with a complete system that does this. Each of the 7,499 proteins is turned into a residue contact graph from its AlphaFold structure and encoded by a graph attention network, the encodings are then passed across the interaction network by a graph convolutional network, and a final predictor pools the encodings of a candidate set and scores it. The model stays compact at just over 58,000 parameters, and it was trained from end to end for 200 epochs on an ordinary CPU.

On the held-out test set it reaches an accuracy of 77.1% with a loss of 0.4911, an ROC-AUC of 0.8692, an average precision of 0.8864, and an F1-score of 0.7900, with recall of 0.8617 sitting above a precision of 0.7294. The test and validation scores line up closely, which gives fair confidence that the model generalises instead of memorising. Read together, the numbers back the idea at the centre of the project, that mixing structural, network and supervisory signals through graph neural networks is a practical route to predicting protein complexes. They also point to obvious next steps, since steadier training and a larger set of labelled complexes are the most promising ways to push the scores higher, and they complete the groundwork from the previous semester by turning it into a model that returns a calibrated answer for any set of proteins drawn from the indexed universe.

9 Future Work

Several extensions would broaden the reach and usefulness of the system:

- **Complex enumeration.** The current model scores a supplied set; coupling it with a candidate-generation procedure—for example, scoring dense sub-graphs of the PPI network or growing complexes greedily around high-scoring seeds—would let it discover complexes across the whole proteome rather than only judging proposed ones.
- **Inductive prediction for new proteins.** Replacing the transductive GCN with an inductive graph-learning scheme would allow the model to embed and score proteins that were not part of the original index, making it directly applicable to newly characterised proteins.
- **Higher-order and hypergraph modelling.** Complexes are intrinsically many-body objects. Modelling them with hypergraph learning, in the spirit of recent hypergraph autoencoders, could capture group-level relationships that pairwise edges miss and further improve accuracy.

References

- [1] J. Jumper, R. Evans, A. Pritzel, et al., “Highly accurate protein structure prediction with AlphaFold,” *Nature*, vol. 596, pp. 583–589, 2021.
- [2] M. Varadi, S. Anyango, M. Deshpande, et al., “AlphaFold Protein Structure Database: massively expanding the structural coverage of protein-sequence space,” *Nucleic Acids Research*, vol. 50, no. D1, pp. D439–D444, 2022. [Online]. Available: <https://alphafold.ebi.ac.uk/>
- [3] P. J. A. Cock, T. Antao, J. T. Chang, et al., “Biopython: freely available Python tools for computational molecular biology and bioinformatics,” *Bioinformatics*, vol. 25, no. 11, pp. 1422–1423, 2009.
- [4] K. Jha, S. Saha, and H. Singh, “Prediction of protein–protein interaction using graph neural networks,” *Scientific Reports*, vol. 12, art. 8360, 2022.
- [5] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph attention networks,” in *Proc. International Conference on Learning Representations (ICLR)*, 2018.
- [6] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *Proc. International Conference on Learning Representations (ICLR)*, 2017.
- [7] J. Lee, I. Lee, and J. Kang, “Self-attention graph pooling,” in *Proc. International Conference on Machine Learning (ICML)*, 2019, pp. 3734–3743.
- [8] M. Fey and J. E. Lenssen, “Fast graph representation learning with PyTorch Geometric,” in *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.
- [9] B. Lai and J. Xu, “Accurate protein function prediction via graph attention networks with predicted structure information,” *Briefings in Bioinformatics*, vol. 23, no. 1, bbab502, 2022.
- [10] S. Xia, et al., “Integration of protein sequence and protein–protein interaction data by hypergraph learning to identify novel protein complexes,” *Briefings in Bioinformatics*, vol. 25, no. 4, 2024.
- [11] J. Zahiri, A. Emamjomeh, S. Bagheri, et al., “Protein complex prediction: a survey,” *Genomics*, vol. 112, no. 1, pp. 174–183, 2020.
- [12] S. Brohee and J. van Helden, “Evaluation of clustering algorithms for protein–protein interaction networks,” *BMC Bioinformatics*, vol. 7, art. 488, 2006.
- [13] G. D. Bader and C. W. V. Hogue, “An automated method for finding molecular complexes in large protein interaction networks,” *BMC Bioinformatics*, vol. 4, art. 2, 2003.
- [14] M. Ashburner, et al., “Gene Ontology: tool for the unification of biology,” *Nature Genetics*, vol. 25, no. 1, pp. 25–29, 2000.
- [15] G. Alanis-Lobato, M. A. Andrade-Navarro, and M. H. Schaefer, “HIPPIE v2.0: enhancing meaningfulness and reliability of protein–protein interaction networks,” *Nucleic Acids Research*, vol. 45, no. D1, pp. D408–D414, 2017.

- [16] M. A. S. Sejan, M. H. Rahman, M. A. Aziz, I. Hameed, M. S. Islam, S. R. Sabuj, and H.-K. Song, "Learning robust node representations via graph neural network and multilayer perceptron classifier," *Mathematics*, vol. 14, no. 4, art. 680, 2026.
- [17] D. Kelesis, D. Fotakis, and G. Paliouras, "Partially trained graph convolutional networks resist oversmoothing," *Machine Learning*, vol. 114, art. 211, 2025.
- [18] W. Hu, B. Liu, J. Gomes, M. Zitnik, P. Liang, V. Pande, and J. Leskovec, "Strategies for pre-training graph neural networks," in *Proc. International Conference on Learning Representations (ICLR)*, 2020.
- [19] S. Chen, C. Shen, Y. Park, and C. E. Priebe, "Graph neural networks powered by encoder embedding for improved node learning," *arXiv preprint arXiv:2507.11732*, 2025.
- [20] K. Jha, S. Saha, and H. Singh, "PPI_GNN: source code for protein–protein interaction prediction using graph neural networks," GitHub repository, 2022. [Online]. Available: https://github.com/JhaKanchan15/PPI_GNN